

AI Assisted Coding Assignment- 11.5

SAI SPURTHI.B

2303A51886

BATCH-30

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Lab Objectives

- Use AI to assist in designing and implementing fundamental data structures in Python.
- Learn how to prompt AI for structure creation, optimization, and documentation.
- Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.
- Enhance code quality with AI-generated comments and performance suggestions.

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

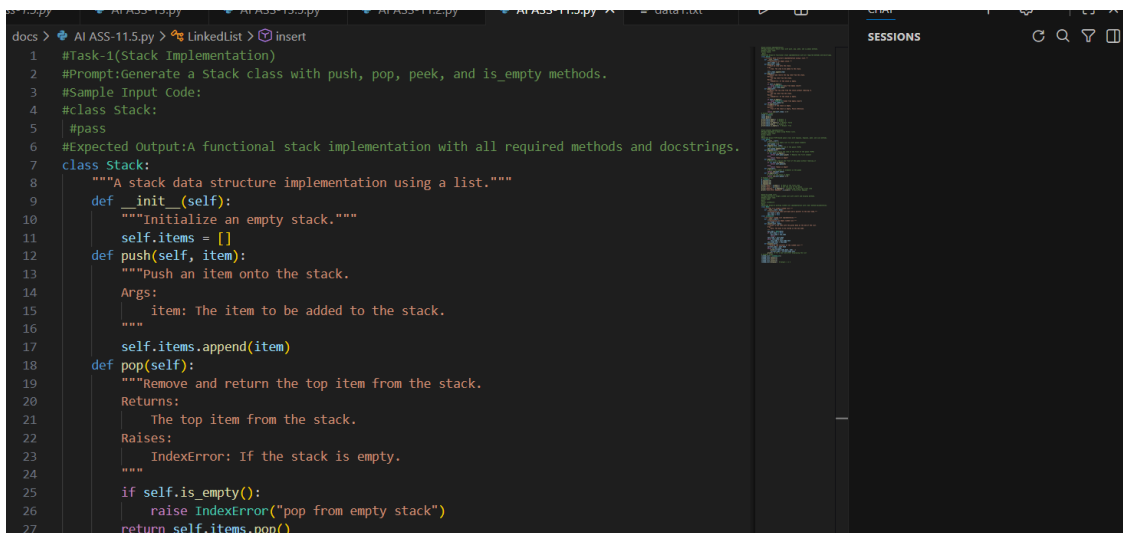
Sample Input Code:

```
class Stack:
```

```
pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.



```
docs > AI ASS-11.5.py > LinkedList > insert
1 #Task-1(Stack Implementation)
2 #Prompt:Generate a Stack class with push, pop, peek, and is_empty methods.
3 #Sample Input Code:
4 #class Stack:
5 #pass
6 #Expected Output:A functional stack implementation with all required methods and docstrings.
7 class Stack:
8     """A stack data structure implementation using a list."""
9     def __init__(self):
10         """Initialize an empty stack."""
11         self.items = []
12     def push(self, item):
13         """Push an item onto the stack.
14         Args:
15             item: The item to be added to the stack.
16         """
17         self.items.append(item)
18     def pop(self):
19         """Remove and return the top item from the stack.
20         Returns:
21             The top item from the stack.
22         Raises:
23             IndexError: If the stack is empty.
24         """
25         if self.is_empty():
26             raise IndexError("pop from empty stack")
27         return self.items.pop()
```

The screenshot shows a VS Code editor with a Python file named `AI ASS-11.5.py`. The code defines a `Stack` class with methods `peek`, `is_empty`, `push`, and `pop`. The `peek` method returns the top item without removing it, `is_empty` checks if the stack is empty, `push` adds an item, and `pop` removes the top item. The terminal output shows the execution of the code, which pushes 1 and 2 onto the stack, peeks at 2, pops it (outputting 2), checks if empty (outputting False), pops 1 (outputting 1), and checks if empty (outputting True).

```
docs > AI ASS-11.5.py > LinkedList > Insert
7 class Stack:
28     def peek(self):
29         """Return the top item from the stack without removing it.
30         Returns:
31             The top item from the stack.
32         Raises:
33             IndexError: If the stack is empty.
34         """
35         if self.is_empty():
36             raise IndexError("peek from empty stack")
37         return self.items[-1]
38     def is_empty(self):
39         """Check if the stack is empty.
40         Returns:
41             True if the stack is empty, False otherwise.
42         """
43         return len(self.items) == 0
44     # Example usage:
45     stack = Stack()
46     stack.push(1)
47     stack.push(2)
48     print(stack.peek()) # Output: 2
49     print(stack.pop())  # Output: 2
50     print(stack.is_empty()) # Output: False
51     print(stack.pop())  # Output: 1
52     print(stack.is_empty()) # Output: True
53
```

Terminal Output:

```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/do
cs/AI ASS-11.5.py"
2
2
False
1
True
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```
docs > AI ASS-11.5.py > LinkedList > Insert
58 #class Queue:
59 #pass
60 #Expected Output:FIFO-based queue class with enqueue, dequeue, peek, and size methods.
61 class Queue:
62     def __init__(self):
63         # Initialize an empty list to store queue elements
64         self.queue = []
65     def enqueue(self, item):
66         # Add an item to the end of the queue (FIFO)
67         self.queue.append(item)
68     def dequeue(self):
69         # Remove and return the item at the front of the queue (FIFO)
70         if not self.is_empty():
71             return self.queue.pop(0) # Removes the first element
72         else:
73             return "Queue is empty"
74     def peek(self):
75         # Return the item at the front of the queue without removing it
76         if not self.is_empty():
77             return self.queue[0]
78         else:
79             return "Queue is empty"
80     def size(self):
81         # Return the number of elements in the queue
82         return len(self.queue)
83     def is_empty(self):
84         # Check if the queue is empty
85         return len(self.queue) == 0
86 # Example usage:
87 q = Queue()
88 q.enqueue(10)
89 q.enqueue(20)
90 q.enqueue(30)
91 print("Peek:", q.peek()) # Peek at the front item
92 print("Size:", q.size()) # Get the size of the queue
93 print("Dequeue:", q.dequeue()) # Remove and return the front item
94 print("Size after dequeue:", q.size()) # Size after dequeue

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Peek: 10
Size: 3
Size: 3
Dequeue: 10
Size after dequeue: 2
```

Tip: Use /create-agent to scaffold a custom agent for your workflow.

+ AIASS-11.Spy
Describe what to build next

Ln 128, Col 34 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation

```
docs > AI ASS-11.5.py > LinkedList > insert
103 pass
104 #Expected Output:A working linked list implementation with clear method documentation.
105 class Node:
106     """A node in a singly linked list."""
107     def __init__(self, data):
108         """Initialize a node with data and a pointer to the next node."""
109         self.data = data
110         self.next = None
111 class LinkedList:
112     """A singly linked list implementation."""
113     def __init__(self):
114         """Initialize an empty linked list."""
115         self.head = None
116     def insert(self, data):
117         """Insert a new node with the given data at the end of the list.
118         Args:
119             data: The data to be stored in the new node.
120         """
121         new_node = Node(data)
122         if self.head is None:
123             self.head = new_node
124             return
125         last_node = self.head
126         while last_node.next:
127             last_node = last_node.next
128         last_node.next = new_node
129     def display(self):
130         """Display the contents of the linked list."""
131         current_node = self.head
132         while current_node:
133             print(current_node.data, end=' ')
134             current_node = current_node.next
135         print() # For a new line after displaying the list
136 # Example usage:
137 linked_list = LinkedList()
138 linked_list.insert(1)
139 linked_list.insert(2)
140 linked_list.insert(3)
141 linked_list.display() # Output: 1 2 3
```

PS C:\Users\HP\Desktop\AI>

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

```
143 # Task Description #4 - Hash Table
144 class HashTable:
145     def __init__(self, size=10):
146         """Initialize hash table with given size"""
147         self.size = size
148         self.table = [[] for _ in range(size)]
149     def _hash(self, key):
150         """Generate hash index"""
151         return hash(key) % self.size
152     def insert(self, key, value):
153         """Insert key-value pair"""
154         index = self._hash(key)
155         for i, (k, v) in enumerate(self.table[index]):
156             if k == key:
157                 self.table[index][i] = (key, value) # update existing key
158                 return
159         self.table[index].append((key, value)) # add new key
160     def search(self, key):
161         """Search value by key"""
162         index = self._hash(key)
163         for k, v in self.table[index]:
164             if k == key:
165                 return v
166         return None
167     def delete(self, key):
168         """Delete key-value pair"""
169         index = self._hash(key)
170         for i, (k, v) in enumerate(self.table[index]):
171             if k == key:
172                 del self.table[index][i]
173                 return True
174         return False
175 # Example usage:
176 hash_table = HashTable()
177 hash_table.insert("name", "Alice")
178 hash_table.insert("age", 30)
179 print(hash_table.search("name"))
180 print(hash_table.search("age"))
181 hash_table.delete("name")
182 print(hash_table.search("name"))
```

Alice
None

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

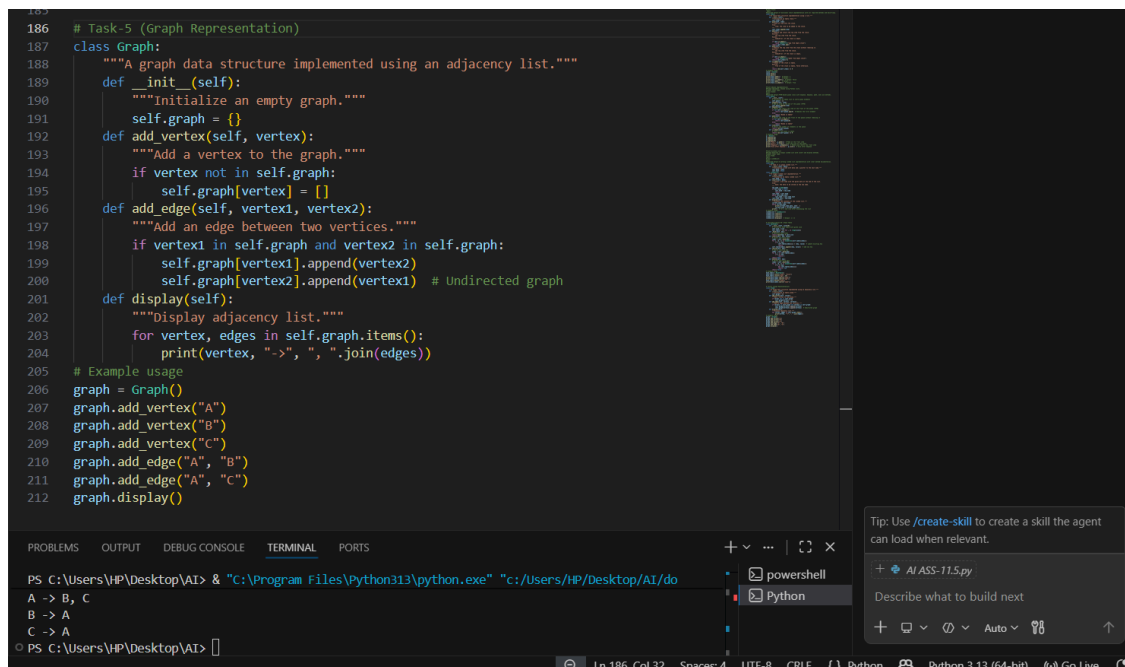
Sample Input Code:

```
class Graph:
```

```
pass
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.



```
186 # Task-5 (Graph Representation)
187 class Graph:
188     """A graph data structure implemented using an adjacency list."""
189     def __init__(self):
190         """Initialize an empty graph."""
191         self.graph = {}
192     def add_vertex(self, vertex):
193         """Add a vertex to the graph."""
194         if vertex not in self.graph:
195             self.graph[vertex] = []
196     def add_edge(self, vertex1, vertex2):
197         """Add an edge between two vertices."""
198         if vertex1 in self.graph and vertex2 in self.graph:
199             self.graph[vertex1].append(vertex2)
200             self.graph[vertex2].append(vertex1) # Undirected graph
201     def display(self):
202         """Display adjacency list."""
203         for vertex, edges in self.graph.items():
204             print(vertex, "->", ", ".join(edges))
205 # Example usage
206 graph = Graph()
207 graph.add_vertex("A")
208 graph.add_vertex("B")
209 graph.add_vertex("C")
210 graph.add_edge("A", "B")
211 graph.add_edge("A", "C")
212 graph.display()
```

PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/do

A -> B, C
B -> A
C -> A

PS C:\Users\HP\Desktop\AI>

Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
215 # Task-6: Smart Hospital Management System - Data Structure Selection
216 # Prompt: A hospital wants to develop a Smart Hospital Management System
217 print("\n" + "-"*80)
218 print("TASK-6: SMART HOSPITAL MANAGEMENT SYSTEM - DATA STRUCTURE SELECTION")
219 print("-"*80)
220 # =====
221 # TABLE 1: FEATURE -> DATA STRUCTURE -> JUSTIFICATION
222 # =====
223 print("\n" + "-"*80)
224 print("TABLE 1: Data Structure Selection for Hospital Features")
225 print("-"*80)
226 table_data = [
227     ("Patient Check-In System", "Queue",
228      "FIFO principle ensures patients are treated in order of arrival."),
229     ("Emergency Case Handling", "Priority Queue",
230      "Critical patients are treated first based on severity level."),
231     ("Medical Records Storage", "Hash Table",
232      "Fast retrieval using patient ID as key with constant time lookup."),
233     ("Doctor Appointment Scheduling", "Binary Search Tree (BST)",
234      "Appointments automatically sorted by time for efficient scheduling."),
235     ("Hospital Room Navigation", "Graph",
236      "Rooms and wards represented as nodes connected by paths.")
237 ]
238
239 print(f"\n{'Feature':<35} | {'Data Structure':<25} | {'Justification'}")
240 print("-" * 110)
241
242 for feature, structure, justification in table_data:
243     print(f"{feature:<35} | {structure:<25} | {justification}")
244
245 # =====
246 # TABLE 2: FUNCTIONAL IMPLEMENTATION
247 # Emergency Case Handling using Priority Queue
248 # =====
249 print("\n" + "-"*80)
250 print("TABLE 2: Implementation - Emergency Case Handling (Priority Queue)")
```

```
252 print("\n" + "-"*80)
253 print("TABLE 2: Implementation - Emergency Case Handling (Priority Queue)")
254 print("-"*80)
255 import heapq
256 class EmergencyCaseManager:
257     """
258     Manages emergency cases using a priority queue.
259     Lower severity number = higher priority.
260     """
261     def __init__(self):
262         """Initialize emergency queue."""
263         self.emergency_queue = []
264         self.patient_counter = 0
265     def add_patient(self, patient_name, severity):
266         """
267         Add patient to emergency queue.
268         severity:
269             1 = Critical
270             2 = Urgent
271             3 = Moderate
272         """
273         self.patient_counter += 1
274         heapq.heappush(self.emergency_queue,
275                       (severity, self.patient_counter, patient_name))
276
277     print(f"Added: {patient_name} (Severity {severity})")
278     def treat_next_patient(self):
279         """Treat next patient based on priority."""
280         if self.emergency_queue:
281             severity, patient_id, patient_name = heapq.heappop(self.emergency_queue)
282             severity_label = {
283                 1: "CRITICAL",
284                 2: "URGENT",
285                 3: "MODERATE"
286             }
287             print(f"Now treating: {patient_name} ({severity_label[severity]})")
288             return patient_name
289         else:
290             print("No patients in queue")
```

```
256 class EmergencyCaseManager:
257     def treat_next_patient(self):
258         else:
259             print("No patients in queue")
260             return None
261     def peek_next_patient(self):
262         """View next patient without removing."""
263         if self.emergency_queue:
264             severity, patient_id, patient_name = self.emergency_queue[0]
265             severity_label = {
266                 1: "CRITICAL",
267                 2: "URGENT",
268                 3: "MODERATE"
269             }
270             print(f"Next patient: {patient_name} ({severity_label[severity]})")
271             return patient_name
272         else:
273             print("Queue empty")
274             return None
275     def queue_size(self):
276         """Return number of waiting patients."""
277         return len(self.emergency_queue)
278     def display_queue(self):
279         """Display all patients in queue."""
280         if not self.emergency_queue:
281             print("Queue empty")
282             return
283         print("\nCurrent Emergency Queue:")
284         severity_map = {
285             1: "CRITICAL",
286             2: "URGENT",
287             3: "MODERATE"
288         }
289         for severity, patient_id, patient_name in sorted(self.emergency_queue):
290             print(f"({patient_name}) (ID:{patient_id}, {severity_map[severity]})")
291
292 # DEMONSTRATION
293 # =====
294 print("\n" + "-"*80)
```

```
docs > AI ASS-11.5.py > ...
322 # =====
323 # DEMONSTRATION
324 # =====
325 print("\n" + "*" * 80)
326 print("DEMONSTRATION: Emergency Case Management System")
327 print("-" * 80)
328 emergency_manager = EmergencyCaseManager()
329 # Add patients
330 emergency_manager.add_patient("John Doe", 3)
331 emergency_manager.add_patient("Jane Smith", 1)
332 emergency_manager.add_patient("Mike Johnson", 2)
333 emergency_manager.add_patient("Sarah Williams", 1)
334 emergency_manager.add_patient("Tom Brown", 2)
335 # Display queue
336 emergency_manager.display_queue()
337 # Peek next patient
338 print()
339 emergency_manager.peak_next_patient()
340 # Treat patients
341 print("\nTreating Patients by Priority:\n")
342 while emergency_manager.queue_size() > 0:
343     emergency_manager.treat_next_patient()
344 print("\n" + "*" * 80)
```

PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI_ASS-11.5.py"

TASK-6: SMART HOSPITAL MANAGEMENT SYSTEM - DATA STRUCTURE SELECTION

TABLE 1: Data Structure Selection for Hospital Features

Feature	Data Structure	Justification
Patient Check-In System	Queue	FIFO principle ensures patients are treated in order of arrival.
Emergency Case Handling	Priority Queue	Critical patients are treated first based on severity level.
Medical Records Storage	Hash Table	Fast retrieval using patient ID as key with constant time lookup.
Doctor Appointment Scheduling	Binary Search Tree (BST)	Appointments automatically sorted by time for efficient scheduling.
Hospital Room Navigation	Graph	Rooms and wards represented as nodes connected by paths.

```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI_ASS-11.5.py"
```

TABLE 2: Implementation - Emergency Case Handling (Priority Queue)

DEMONSTRATION: Emergency Case Management System

Added: John Doe (Severity 3)
Added: Jane Smith (Severity 1)
Added: Mike Johnson (Severity 2)
Added: Sarah Williams (Severity 1)
Added: Tom Brown (Severity 2)

Current Emergency Queue:
Jane Smith (ID:2, CRITICAL)
Sarah Williams (ID:4, CRITICAL)
Mike Johnson (ID:3, URGENT)
Tom Brown (ID:5, URGENT)
John Doe (ID:1, MODERATE)

Next patient: Jane Smith (CRITICAL)

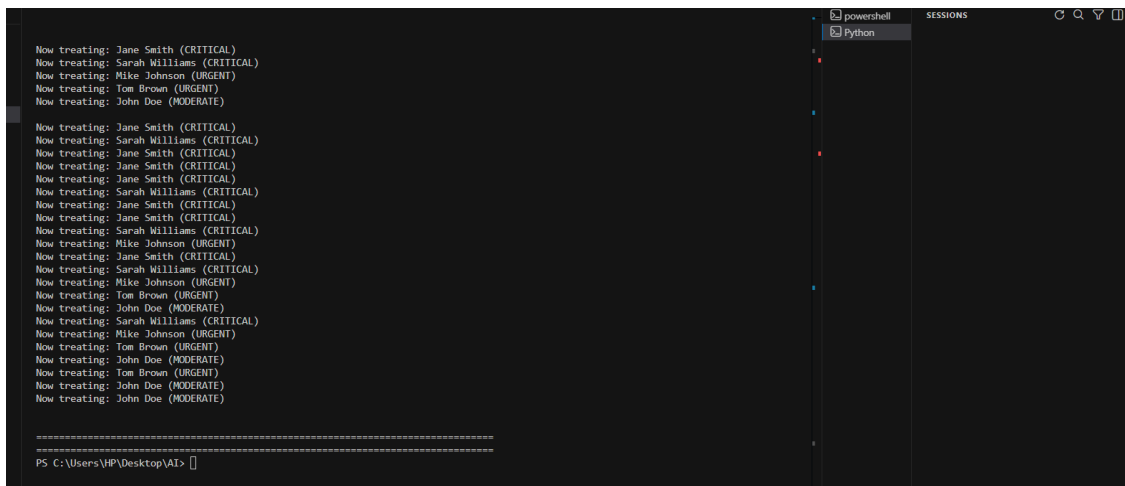
Treating Patients by Priority:

Now treating: Jane Smith (CRITICAL)
Now treating: Sarah Williams (CRITICAL)
Now treating: Mike Johnson (URGENT)
Now treating: Tom Brown (URGENT)
Now treating: John Doe (MODERATE)

PS C:\Users\HP\Desktop\AI>

Now treating: Jane Smith (CRITICAL)
Now treating: Sarah Williams (CRITICAL)
Now treating: Mike Johnson (URGENT)
Now treating: Tom Brown (URGENT)
Now treating: John Doe (MODERATE)

Now treating: Jane Smith (CRITICAL)
Now treating: Sarah Williams (CRITICAL)
Now treating: Jane Smith (CRITICAL)
Now treating: Jane Smith (CRITICAL)



```
Now treating: Jane Smith (CRITICAL)
Now treating: Sarah Williams (CRITICAL)
Now treating: Mike Johnson (URGENT)
Now treating: Tom Brown (URGENT)
Now treating: John Doe (MODERATE)

Now treating: Jane Smith (CRITICAL)
Now treating: Sarah Williams (CRITICAL)
Now treating: Jane Smith (CRITICAL)
Now treating: Jane Smith (CRITICAL)
Now treating: Jane Smith (CRITICAL)
Now treating: Sarah Williams (CRITICAL)
Now treating: Jane Smith (CRITICAL)
Now treating: Jane Smith (CRITICAL)
Now treating: Sarah Williams (CRITICAL)
Now treating: Sarah Williams (CRITICAL)
Now treating: Mike Johnson (URGENT)
Now treating: Jane Smith (CRITICAL)
Now treating: Sarah Williams (CRITICAL)
Now treating: Mike Johnson (URGENT)
Now treating: Tom Brown (URGENT)
Now treating: John Doe (MODERATE)
Now treating: Sarah Williams (CRITICAL)
Now treating: Mike Johnson (URGENT)
Now treating: Tom Brown (URGENT)
Now treating: John Doe (MODERATE)
Now treating: John Doe (MODERATE)
Now treating: John Doe (MODERATE)

=====
PS C:\Users\HP\Desktop\AI>
```

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.

- A functional Python program implementing the chosen feature with comments and docstrings.

```

347 # Task-7: Smart City Traffic Control System
348 # =====
349 # Queue Implementation for Traffic Signal
350 # =====
351 class Queue:
352     """Queue used to manage vehicles waiting at a traffic signal."""
353     def __init__(self):
354         """Initialize an empty queue."""
355         self.queue = []
356     def enqueue(self, vehicle):
357         """Add a vehicle to the queue."""
358         self.queue.append(vehicle)
359         print(f"Vehicle {vehicle} added to the queue.")
360     def dequeue(self):
361         """Remove the first vehicle from the queue."""
362         if self.is_empty():
363             return "No vehicles waiting at the signal."
364         return self.queue.pop(0)
365     def peek(self):
366         """View the first vehicle in the queue."""
367         if self.is_empty():
368             return "No vehicles waiting at the signal."
369         return self.queue[0]
370     def size(self):
371         """Return number of vehicles waiting."""
372         return len(self.queue)
373     def is_empty(self):
374         """Check if queue is empty."""
375         return len(self.queue) == 0
376 # =====
377 # TABLE: Feature -> Data Structure -> Justification
378 # =====
379 print("\nFeature -> Chosen Data Structure -> Justification\n")
380 feature_table = [
381     ("Traffic Signal Queue", "Queue",
382      "Vehicles are processed in arrival order using FIFO principle."),
383     ("Emergency Vehicle Priority", "Priority Queue",
384      "Emergency vehicles like ambulances must be handled first."),
385     ("Vehicle Registration Lookup", "Hash Table",
386      "Vehicle number can be used as a key for fast lookup."),
387     ("Road Network Mapping", "Graph",
388      "Intersections are nodes and roads are edges in a network."),
389     ("Parking Slot Availability", "Linked List",
390      "Parking slots can be dynamically updated easily.")
391 ]

```

```

392 for feature in feature_table:
393     print(f"{feature[0]} -> {feature[1]} -> {feature[2]}")
394 # =====
395 # Demonstration Program
396 # =====
397 print("\n--- Traffic Signal Queue Demonstration ---")
398 signal_queue = Queue()
399 # Vehicles arrive at signal
400 signal_queue.enqueue("Car1")
401 signal_queue.enqueue("Car2")
402 signal_queue.enqueue("Car3")
403 print(f"Number of vehicles waiting: {signal_queue.size()}")
404 print(f"Next vehicle to pass: {signal_queue.peek()}")
405 print(f"Vehicle passed the signal: {signal_queue.dequeue()}")
406 print(f"Remaining vehicles in queue: {signal_queue.queue}")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "C:/Users/HP/Desktop/AI/docs/AI_ASS-11.5.py"

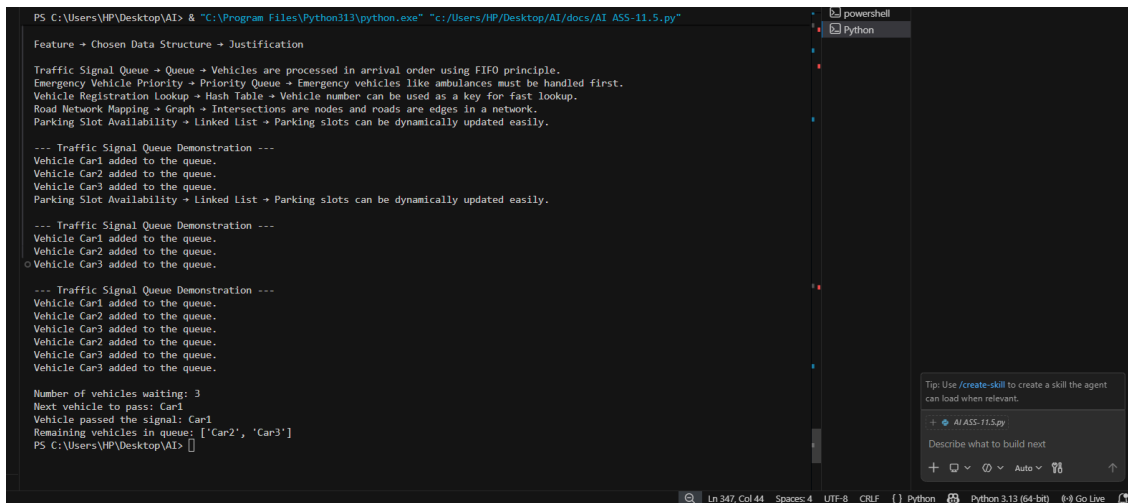
Feature -> Chosen Data Structure -> Justification

Traffic Signal Queue -> Queue -> Vehicles are processed in arrival order using FIFO principle.
Emergency Vehicle Priority -> Priority Queue -> Emergency vehicles like ambulances must be handled first.
Vehicle Registration Lookup -> Hash Table -> Vehicle number can be used as a key for fast lookup.
Road Network Mapping -> Graph -> Intersections are nodes and roads are edges in a network.
Parking Slot Availability -> Linked List -> Parking slots can be dynamically updated easily.

--- Traffic Signal Queue Demonstration ---
Vehicle Car1 added to the queue.
Vehicle Car2 added to the queue.
Vehicle Car3 added to the queue.
Parking Slot Availability -> Linked List -> Parking slots can be dynamically updated easily.

--- Traffic Signal Queue Demonstration ---
Vehicle Car1 added to the queue.

Ln 347, Col 44 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live



```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI_ASS-11.5.py"

Feature -> Chosen Data Structure -> Justification

Traffic Signal Queue -> Queue -> Vehicles are processed in arrival order using FIFO principle.
Emergency Vehicle Priority -> Priority Queue -> Emergency vehicles like ambulances must be handled first.
Vehicle Registration Lookup -> Hash Table -> Vehicle number can be used as a key for fast lookup.
Road Network Mapping -> Graph -> Intersections are nodes and roads are edges in a network.
Parking Slot Availability -> Linked List -> Parking slots can be dynamically updated easily.

--- Traffic Signal Queue Demonstration ---
Vehicle Car1 added to the queue.
Vehicle Car2 added to the queue.
Vehicle Car3 added to the queue.
Parking Slot Availability -> Linked List -> Parking slots can be dynamically updated easily.

--- Traffic Signal Queue Demonstration ---
Vehicle Car1 added to the queue.
Vehicle Car2 added to the queue.
Vehicle Car3 added to the queue.

--- Traffic Signal Queue Demonstration ---
Vehicle Car1 added to the queue.
Vehicle Car2 added to the queue.
Vehicle Car3 added to the queue.
Vehicle Car2 added to the queue.
Vehicle Car3 added to the queue.
Vehicle Car3 added to the queue.

Number of vehicles waiting: 3
Next vehicle to pass: Car1
Vehicle passed the signal: Car1
Remaining vehicles in queue: ['Car2', 'Car3']
PS C:\Users\HP\Desktop\AI>
```

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
408
409 # Task-8: Smart E-Commerce Platform - Data Structure Challenge
410 # =====
411 # Feature → Data Structure → Justification Table
412 # =====
413 feature_table = [
414     ("Shopping Cart Management", "Deque",
415      "Deque allows adding and removing products efficiently from both ends."),
416     ("Order Processing System", "Queue",
417      "Queue follows FIFO principle so orders are processed in the order they are placed."),
418     ("Top-Selling Products Tracker", "Priority Queue",
419      "Priority queue helps track products ranked by sales count."),
420     ("Product Search Engine", "Hash Table",
421      "Hash table enables fast product lookup using product ID."),
422     ("Delivery Route Planning", "Graph",
423      "Graph models warehouses and delivery locations as nodes and routes as edges.")
424 ]
425 print("Feature → Chosen Data Structure → Justification\n")
426 for feature in feature_table:
427     print(f"{feature[0]} → {feature[1]} → {feature[2]}")
428 # =====
429 # Order Processing System Implementation using Queue
430 # =====
431 class Queue:
432     """
433     Queue class used to process orders in FIFO order.
434     """
435     def __init__(self):
436         """Initialize an empty order queue."""
437         self.queue = []
438     def enqueue(self, order_id):
439         """Add an order to the queue."""
440         self.queue.append(order_id)
441         print(f"Order {order_id} added to the queue.")
442     def dequeue(self):
443         """Process the first order in the queue."""
444         if self.is_empty():
445             return "No orders to process."
446         return self.queue.pop(0)
447     def peek(self):
448         """View the next order without removing it."""
449         if self.is_empty():
450             return "No orders to process."
451         return self.queue[0]
452     def size(self):
453         """Return number of orders waiting."""
454         return len(self.queue)
455     def is_empty(self):
456         """Check if queue is empty."""
457         return len(self.queue) == 0
458 # =====
459 # Demonstration Program
460 # =====
461 if __name__ == "__main__":
462     order_queue = Queue()
463     # Add orders
464     order_queue.enqueue("Order_001")
465     order_queue.enqueue("Order_002")
466     order_queue.enqueue("Order_003")
467     print(f"Number of orders in queue: {order_queue.size()}")
468     print(f"Next order to process: {order_queue.peek()}")
469     print(f"Processing order: {order_queue.dequeue()}")
470     print(f"Remaining orders: {order_queue.queue}")
```

```
docs > AI ASS-11.5.py > ...
431 class Queue:
432     self.queue = []
433     def enqueue(self, order_id):
434         """Add an order to the queue."""
435         self.queue.append(order_id)
436         print(f"Order {order_id} added to the queue.")
437     def dequeue(self):
438         """Process the first order in the queue."""
439         if self.is_empty():
440             return "No orders to process."
441         return self.queue.pop(0)
442     def peek(self):
443         """View the next order without removing it."""
444         if self.is_empty():
445             return "No orders to process."
446         return self.queue[0]
447     def size(self):
448         """Return number of orders waiting."""
449         return len(self.queue)
450     def is_empty(self):
451         """Check if queue is empty."""
452         return len(self.queue) == 0
453 # =====
454 # Demonstration Program
455 # =====
456 if __name__ == "__main__":
457     order_queue = Queue()
458     # Add orders
459     order_queue.enqueue("Order_001")
460     order_queue.enqueue("Order_002")
461     order_queue.enqueue("Order_003")
462     print(f"Number of orders in queue: {order_queue.size()}")
463     print(f"Next order to process: {order_queue.peek()}")
464     print(f"Processing order: {order_queue.dequeue()}")
465     print(f"Remaining orders: {order_queue.queue}")
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI_ASS-11.5.py"
Feature + Chosen Data Structure + Justification

Shopping Cart Management + Deque + Deque allows adding and removing products efficiently from both ends.
Order Processing System + Queue + Queue follows FIFO principle so orders are processed in the order they are placed.
Top-Selling Products Tracker + Priority Queue + Priority queue helps track products ranked by sales count.
Product Search Engine + Hash Table + Hash table enables fast product lookup using product ID.
Delivery Route Planning + Graph + Graph models warehouses and delivery locations as nodes and routes as edges.
Order Order_001 added to the queue.
Order Order_002 added to the queue.
Order Order_003 added to the queue.

Order Order_001 added to the queue.
Order Order_002 added to the queue.
Order Order_003 added to the queue.

Order Order_003 added to the queue.

Number of orders in queue: 3
Next order to process: Order_001
Processing order: Order_001
Remaining orders: ['Order_002', 'Order_003']
PS C:\Users\HP\Desktop\AI>
```

Tip: Use /create-skill to create a skill the agent can load when relevant.

+ AI_ASS-11.5.py

Describe what to build next